

Fast feature extraction for machining applications

M C Fields and D C Anderson*

An algorithmic method for feature shape extraction from solid geometric models is presented. The method is based on a boundary representation graph and categorizes sets of faces, based on the topology and geometry of the set, into well defined, nonintersecting, machining cavity feature classes. Shapes within a class can vary in topology and geometry but share many machining-oriented characteristics. Fast linear time recognition algorithms are also presented. The paper concludes with a discussion of a prototype feature recognition implementation.

Keywords: features, feature recognition, feature extraction

Solid geometric modelling plays an integral part in mechanical design and in engineering applications that depend on the results of these designs. Examples of these applications include computer aided process planning and finite element analysis. The solid model is usually represented in a graph format in which low level features of the model, such as vertices, edges, and faces, are represented explicitly. However, engineering applications often require knowledge of higher level features, such as cavities (e.g. holes and pockets), for machining process planning. Methods for high level feature recognition have been studied for many years. The most prominent methods focus on features in a machining domain used in computer aided process planning systems.

In this paper we present an efficient method for general feature shape recognition. Like many of the previous methods for automatic feature recognition, we concentrate on the domain of features that commonly occur in machined parts. However, unlike many previous methods, this method does not rely on feature set enumeration or computationally intractable search algorithms. The method categorizes sets of faces on the surface of the part into classes of general machining domain features where shapes within each class can vary in topology and geometry but share many machining oriented characteristics. Fast linear time recognition algorithms are presented for each feature class. See, for example, *Figure 1*, which depicts a nontrivial machined part¹. The

methods and algorithms presented in this paper recognize 13 machining oriented features. The prototype feature recognizer discussed in the sixth section of the paper requires less than 1 s to identify these features.

In the following section, a brief survey of previous and related work is presented in order to put the problems involved into perspective. In the third section, preliminary definitions are presented. The fourth section presents a classification method for general machining domain features, and the fifth section presents recognition algorithms for this general method along with a formal analysis of the complexity. The sixth section presents a prototype implementation along with some examples and experimental data. The seventh section presents conclusions along with the open problems and future work.

BACKGROUND

Webster's Ninth New Collegiate Dictionary defines a feature of an object as a 'prominent part or characteristic' of that object². Dixon *et al.*³ describe a feature as 'any geometric form or entity that is used in reasoning in one or more design or manufacturing activities'. In this work a feature is defined as a set of adjacent faces of a part that correspond to shapes that commonly occur on the surface of machined parts, for example pockets, holes, and protrusions. With this definition in mind, feature recognition is defined as the grouping or partitioning of the set of faces on the surface of the part such that each face set represents a feature.

Solid modelling systems represent objects in a variety of formats. A popular format, referred to as a boundary representation, or B-rep, describes the solid by defining its boundary using a graph embedded in 3D space. In this work a boundary representation is used to describe the part.

Given a B-rep for the feature, a graph match can be performed between the feature B-rep and a subgraph of the part B-rep. This is essentially the subgraph isomorphism problem that is known to be NP-complete⁴. However, there are polynomial time algorithms that involve special cases of subgraph isomorphism. One such method is presented in Reference 5, in which a technique to compute the congruency of polyhedra and portions of polyhedra is given. It shows how the graph isomorphism technique of Hopcroft and Tarjan⁶ can be applied if the graph is mapped to the surface of a solid object meeting

Advanced Manufacturing Department, United Technologies Carrier, Building TR-4, Syracuse, NY 13221, USA

*School of Mechanical Engineering, Purdue University, 1291 Potter Building Room 134, West Lafayette, IN 47907-1291, USA

Paper received: 18 May 1993. Revised: 2 October 1993

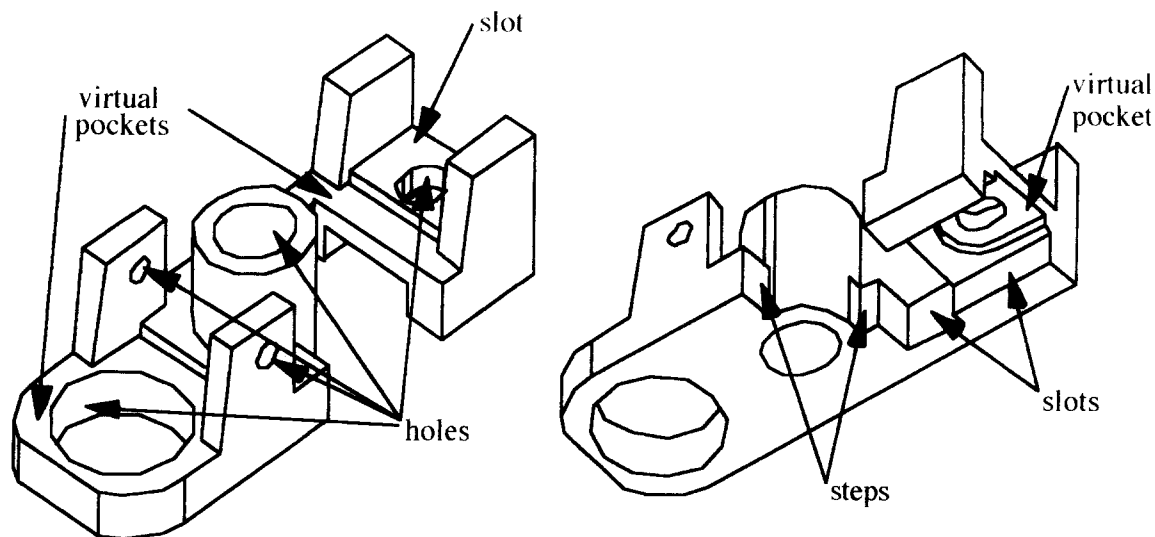


Figure 1 Benchmarking part

certain restrictions. The congruency of two solids is computed in $O(n \log n)$ time, and partial congruency is computed in $O(n^2)$ time, where n is the number of edges on the surface of the solid. However, this technique requires that, for two solids to be designated congruent, they must be topologically and geometrically congruent with respect to a linear coordinate transformation.

There are many other methods based on matching techniques presented in the literature that are specifically designated for feature recognition. Two of these are given in References 7 and 8. These methods define a finite set of feature pattern graphs *a priori*. Each feature pattern graph contains the necessary topological and geometrical information to describe a feature. The feature graphs are matched against a subgraph of the graph that describes the part. Heuristic methods based on graph matching are employed to improve the speed of the search. In the worst case, however, the methods still require exponential time.

The matching method presented in Reference 9 has the advantage of a more efficient subgraph matching algorithm, and an interesting technique in which a perfect match at the boundaries of the feature pattern is not always required. The algorithm becomes more efficient by implicitly restricting the types of desired feature pattern graph that can be used in the search. From a formal complexity point of view, where N_f designates the number of faces in the pattern and N_s designates the number of faces on the solid object, the algorithm has a worst case time complexity of $O(N_f^2 \times N_s)$.

All matching methods are restricted to predefining and matching against a finite set of feature pattern graphs, and thus they cannot recognize general features of varying topology, for example n -sided pockets. This is also important in the case of arbitrary feature interactions, for example when two pockets intersect, or if only a polygonal approximation to the object is available. Also, because this enumerated set can be very large, even the most efficient graph matching algorithm will not yield an efficient overall method. Finally, all matching methods fail to partition the features uniquely. For example, a common machining domain feature is a step. A step can

be described as a pair of adjacent faces, meeting at a concave angle, that cause a stepped shape on the surface of a part. Many matching methods will recognize a side and the bottom of a pocket also as a step. This is true when the pattern subgraph for a step is simply two faces meeting at a concave angle, as discussed in References 7 and 9 and others. This results in redundant features, and therefore a time consuming method is required to maintain a proper partition during the search. Alternatively, the redundant features can be removed, after the fact, by a computationally expensive post-processing phase.

Many methods are described in the literature that are not based on a graph matching approach. One of the first such methods is presented in Reference 10, in which logic programming techniques are used for pattern matching. The method works by building a set of rules defining features and using an inference engine to search for the features in the logic space. However, the complexity of the technique is worst case exponential and thus inefficient.

In a different approach based on web and graph grammars, general features are recognized using parsing. This approach is taken in References 11 and 12. Although these methods look very promising because of their ability to describe general feature classes, an efficient solution based on these methods has not been reported.

In Reference 13, an edge-face graph is computed from the original B-rep and features are extracted by computing the biconnected and triconnected components of the edge-face graph. This method has a worst case complexity of $O(m^2)$ where m is the number of vertices, edges, faces, and loops on the surface of the object. This method is very robust because it depends only on the topology of the solid object. However, only features that are connected to a single face or to two faces can be recognized using this technique, and thus the solution cannot recognize some simple features, such as through slots, that are connected to more than two faces. More recently, Reference 14 has expanded on this basic idea by using a PROLOG program to find the biconnected and triconnected components. There are many other

methods that focus on heuristic boundary representation algorithms for feature recognition. Each of these exploits special characteristics of the boundary representation to identify features. These methods are described in, for example, References 15–24.

In Reference 25, a method is presented that searches for *hints* within the B-rep and tries to construct features from these. However, because of the uncertainty involved in heuristic searching, the method cannot be formalized and therefore no argument of completeness or efficiency is presented. Another method very similar to this is presented in Reference 26.

These feature recognition techniques can be categorized by the underlying search algorithms that are employed. The most prominent methods involve some variation of a graph matching technique as discussed in References 7 and 9, mentioned above. The method presented in Reference 13 uses a graph-based approach but no matching is employed. Finally, References 25 and 26 present examples of methods that can heuristically recognize general features. The graph-based approaches have the advantage of being more easily formalized and thus analysed, but in the previous results an efficient method with the ability to recognize general features has not been found.

There are many other feature recognition methods that exploit the volumetric properties of features and the difference between the part boundary and the convex hull of the part. The most recent or notable of these are the methods developed by Wang²⁷, Woo²⁸, Kim²⁹ and Waco and Kim³⁰, and Ferreira and Hinduja³¹.

PRELIMINARY DEFINITIONS

This section presents preliminary definitions that will be used in the rest of the paper. First, a boundary representation graph is defined. Then some minor restrictions are placed on properties of the solid model to exclude pathological cases. Finally, a simplification of the B-rep called the *oriented face adjacency graph* (OFAG) is presented along with the definitions of some of the terms that will be used in the subsequent feature face set descriptions.

Definition 1 (boundary graph): The *boundary graph* of a solid object S is a graph $BR = (V, E, F)$, where V is the set of vertices $\langle x, y, z \rangle \in \mathbb{R}^3$ of S , E is the set of edges of S , and F is the set of faces of S . Each edge e is represented as a 3-tuple $e = \langle v_i, v_t, L \rangle$, where $v_i \in V$ is the initial vertex, $v_t \in V$ is the terminal vertex, and L is the set of faces adjacent to e . Each element f of F is represented as an ordered list of ordered edge cycles, an orientation, and an equation that defines the surface in $\mathbb{R}^3$. The first cycle in the list is called the *exterior edge cycle* which bounds the face on the exterior. The other zero or more cycles in the list are called *interior edge cycles*. These edge cycles define holes in the face. Intuitively, f is the surface defined by the equation and embedded within these edge cycles.

The boundary representation graph is required to be connected, and therefore a solid object that is made of two or more parts or shells is excluded. The object is required to be manifold, i.e. the object cannot have pinch points, dangling edges or dangling faces. No face on the object can be self-adjacent. For example, in the case in

which the representation of a complete cylindrical face includes a seam parallel to the axis of the cylinder, this face will be self-adjacent, and thus would have to be split into two faces. Finally, the B-rep of the object must be minimal, i.e. two faces are never used where one would be sufficient unless the face would become self-adjacent. For example, a connected planar surface of an object must be modelled with one face. With this restriction, no smooth adjacencies between planar faces can exist. In the case of smooth adjacencies involving curved surfaces, 2nd-order conditions must be applied to classify these adjacencies as convex or concave.

Definition 2 (exterior face): The face $f' \in F$ is *exterior* with respect to $f \in F$ if f' and f are adjacent and there exists an edge of adjacency e for f' and f that is on the exterior edge cycle of f .

A simplification of the boundary graph called the *oriented face adjacency graph* is presented next. Informally, the oriented face adjacency graph for a solid object S has a node for every face of S and has two labelled directed arcs between nodes f_i and f_j if the faces of S corresponding to f_i and f_j are adjacent. Each edge carries two labels, one describing whether the adjacency is convex or concave, and one describing whether the adjacency is interior or exterior with respect to one of the faces. As its name implies, this graph is similar to the face adjacency graph that was first introduced in Reference 24. Many graph-based feature recognition approaches have employed a variant of the face adjacency graph⁷.

Definition 3 (oriented face adjacency graph): The *oriented face adjacency graph* $G = (F, E)$ of the solid object S is a simplification of the boundary graph BR of S in which F is the set of nodes of type face and E is a set of elements of type edge. Face nodes in F have a one-to-one correspondence with the face nodes of BR . In addition, each face node has an associated label which is discussed in detail below. For each edge on the solid S , there exist two corresponding edges in E that take the form of tuples $\langle f_i, f_j, l_1, l_2 \rangle$, where $f_i \in F$, $f_j \in F$, $l_1 \in \{+, -\}$ and $l_2 \in \{\text{int}, \text{ext}\}$. The label l_1 in $\langle f_i, f_j, l_1, l_2 \rangle$ will be $-$ (respectively $+$) if the faces on the boundary of S corresponding to f_i and f_j have a concave (respectively convex) adjacency. The label l_2 in $\langle f_i, f_j, l_1, l_2 \rangle$ will be ext (respectively int) if the face corresponding to f_j is (respectively is not) exterior with respect to the face corresponding to f_i .

For example, if f_i and f_j correspond to faces on S that have a concave adjacency, and if the face of S corresponding to f_j is exterior with respect to the face of S corresponding to f_i , then $\langle f_i, f_j, -, \text{ext} \rangle \in E$. Similarly, if f_i and f_j correspond to faces on S that have a convex adjacency, and the face corresponding to f_j is interior with respect to the face corresponding to f_i , then $\langle f_i, f_j, +, \text{int} \rangle \in E$. Other combinations of labels exist. Finally, we require that, just as the edges in the exterior cycle and each interior cycle of some face of the boundary graph BR are ordered, so are the corresponding edges in E .

An example of the oriented face adjacency graph for a simple cube is presented in Figure 2. Faces f_1 and f_2 are convex, and each is exterior with respect to the other.

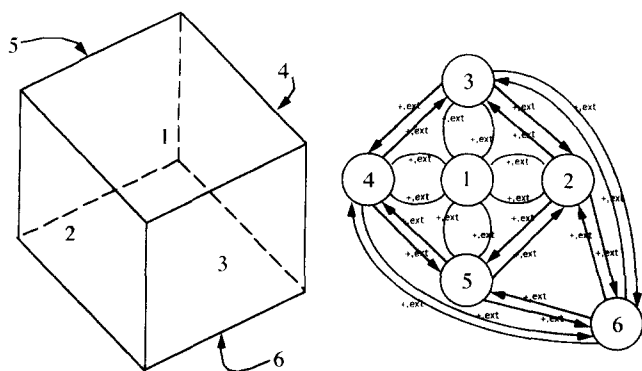


Figure 2 Simple cube and corresponding oriented face adjacency graph
[The numbers refer to faces.]

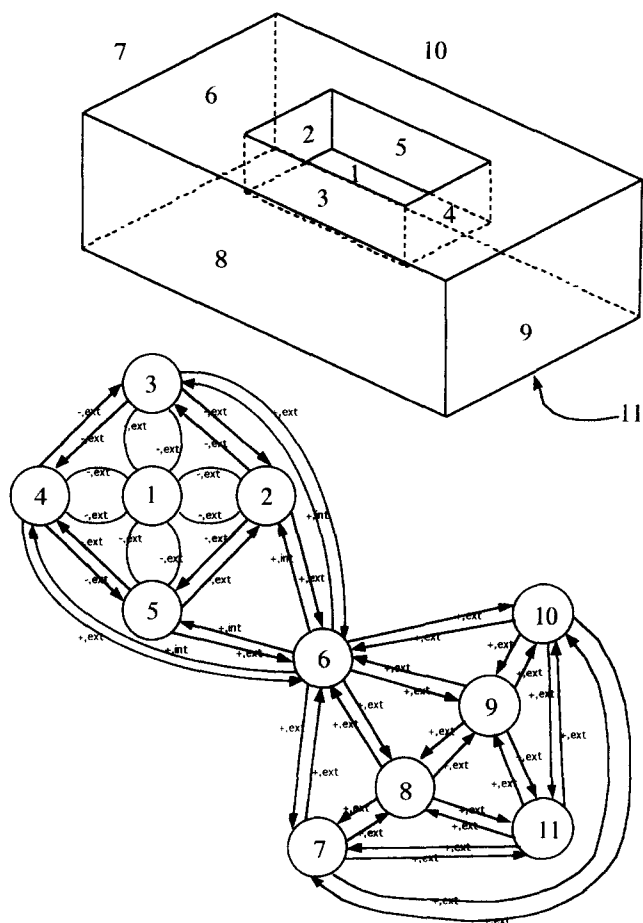


Figure 3 Solid object and its oriented face adjacency graph

Therefore, $\langle f_1, f_2, +, \text{ext} \rangle$ and $\langle f_2, f_1, +, \text{ext} \rangle$ are arcs in the oriented face adjacency graph. Also note that, since f_2, f_3, f_4 and f_5 are adjacent to f_1 , then the arcs incident from f_1 to those face nodes have an implicit ordering that corresponds to the order in which the face nodes occur around face f_1 . Figure 3 presents another example of an oriented face adjacency graph of a part with a pocket.

The face nodes of the OFAG are labelled with respect to the convexity or concavity of the externally adjacent faces and the sequence in which these faces occur around the exterior of the face node. Labels range from *opened*, where all the faces adjacent on the exterior loop of the

face node form a convex angle with the face node, to *closed*, where all these faces form a concave angle with the face node.

Three other labels exist: *unbounded*, *through*, and *bounded*. To understand how these labels are applied, the sequence of convex and concave edges occurring around the exterior loop of the face node must be studied. What are important are the transitions from a sequence of convex edges to a sequence of concave edges. If there are three or more of these transitions from convex to concave, then the face node takes the label *unbounded*. If there are two transitions from convex to concave, then the face node is labelled *through*. Finally, if there is only one transition from convex to concave, then the face node is labelled *bounded*.

For example, refer to the simple polyhedral object depicted in Figure 4. Let us call an edge *convex* and *exterior* if, given a face node f_i , the edge is on the exterior loop that bounds f_i , and the edge is formed by the convex intersection of f_i with the corresponding adjacent face. In Figure 4, face f_1 is labelled *unbounded* because, starting at an exterior convex edge and counting the number of transitions from a convex edge to a concave edge as we move around the exterior of f_1 , three transitions are found. Similarly, f_2 is labelled *through* because two convex to concave transitions are found. Face f_3 is labelled *bounded* because one transition from convex to concave is found.

Additionally, note that face f_4 takes a *closed* label because all faces externally adjacent to f_4 form a concave angle with f_4 . Finally, note that f_5 is labelled *opened* because all externally adjacent faces are convex to f_5 .

Given the graph structure and face and edge labelling described above, a search operator can be defined. This operator, called the *merge* operator because it merges or groups sets of faces together, essentially computes connected components with respect to certain edges and faces within the OFAG. These connected components will eventually represent the features.

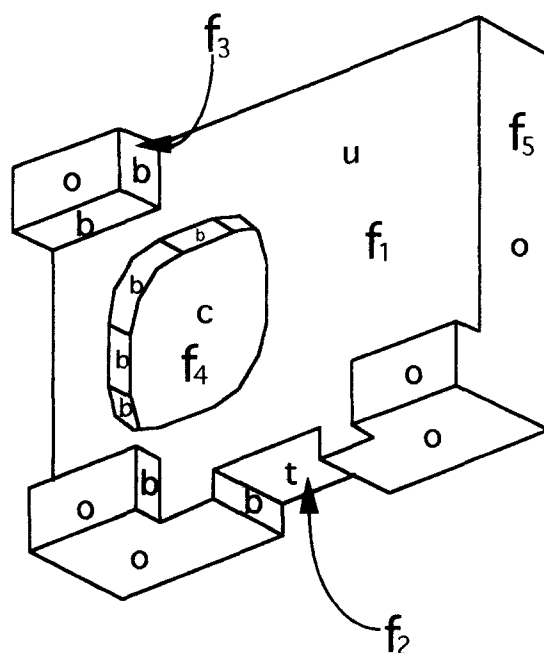


Figure 4 Example of labelling scheme
[o: opened, u: unbounded, t: through, b: bounded, c: closed.]

The merge operator performs a depth-first search in the OFAG. It requires that three parameters be specified: the first describes the acceptable faces based on face labels, the second describes what type of edges (convex, concave, interior or exterior) should be traversed during the search, and the third tells the merge operator the maximum depth from the starting face to search in the OFAG. In addition, face labels that prescribe acceptable starting faces must also be specified.

To begin the search, an acceptable face is found, and a face node set is initialized to include this starting face only. Then, in turn, the merge operator is invoked, and the search is performed, starting from this face, by moving from face to face in the OFAG in a depth-first manner. Only edges that are labelled as directed are traversed during the search. The search is continued from a face if and only if the face node is labelled as prescribed, and if the search depth has not been exceeded. As each face node is visited it is added to the face node set if it is labelled as prescribed. Also, as each face is visited, it is marked so that it will not be considered again in the search. The resulting face node set will ultimately represent the feature. The search algorithm and its implementation is discussed in detail in the fifth section.

Different types of feature can be described by specifying different face and edge types, by specifying different search depths, and by specifying the type of face node from which the search should start. For example, refer to the part and the corresponding OFAG depicted in Figure 3. Note that f_1 would be labelled *closed*, and f_2 , f_3 , f_4 and f_5 labelled *bounded*. With face f_1 as the starting face, and considering only edges labelled concave and external and only faces labelled *bounded*, and allowing an unlimited search depth, the merge operator will compute the face set $\{f_1, f_2, f_3, f_4, f_5\}$. This face set corresponds to what is commonly referred to as a pocket feature.

MACHINING DOMAIN FEATURES

The following definitions define five classes of machining domain feature. Each definition refers to the merge operator and defines a feature class by describing the acceptable face types (based on the face labels), the acceptable edge types, the recursive search depth, and the type of face from which to start. Additionally, some other restrictions are placed on the merged face sets so that the results agree more closely with what we intuitively perceive to be features.

A prominent machining domain feature type is the depression. Depressions, also referred to as pockets or cavities, are intuitively characterized as having a principal face bounded by an arbitrary number of side faces. Depressions are categorized in this work into three classes. The following definitions are general in that they classify most depressions regardless of the number of faces involved, and regardless of interactions with other feature shapes.

Definition 4 (type 1 depression): A type 1 depression is a subset of the faces of the oriented face adjacency graph formed by applying the merge operator starting at a face node labelled *closed*, specifying a recursive search depth of 1, and considering only bounded faces and concave external edges. Additionally, for the resulting face set to

be considered a type 1 depression, the set must include at least two bounded faces. Note that type 1 depressions can be further subclassified into those with cylindrical sides (e.g. blind holes) and others (e.g. pockets).

Definition 5 (type 2 depression): A type 2 depression is a subset of the faces of the OFAG formed by applying the merge operator starting at a bounded face, specifying an unlimited recursive depth, and considering only bounded faces and concave external edges. Additionally, there must be one bounded face that is adjacent to every other bounded face in the resulting face set, and it is required that this face be not externally adjacent and concave to a face that has a label that is different from *bounded*. This face forms what might be called the bottom of the type 2 depression.

The definition for type 2 depressions requires that the merged face node set must have a special face that is referred to as the bottom face. In some cases, the faces of the merged face node set have to be split or regrouped into smaller sets in order to meet this requirement. The recognition algorithms in the next section perform this partitioning by iteratively choosing the face in the merged face node set with the highest number of concave adjacencies.

Definition 6 (step): A step is an instance of a type 2 depression that only has two face nodes.

Definition 7 (type 3 depression): A type 3 depression is a subset of the faces of the OFAG formed by applying the merge operator starting at a through face or an unbounded face, specifying a recursive depth of 1, and considering only bounded faces and concave external edges. Additionally, it is required that there be at least one bounded face in the resulting face set.

Definition 8 (slot): A slot is an instance of a type 3 depression in which the starting face is labelled *through*. Additionally, it is required that the two side faces of the B-rep corresponding to the bounded face nodes must have opposite pointing normal vectors.

As an example, refer to Figure 3, which illustrates a part with a simple type 1 depression. Face node f_1 is labelled *closed*. Face nodes f_2 , f_3 , f_4 and f_5 are each labelled *bounded*. As the definition of type 1 depressions states, when the search for a type 1 depression is performed, f_1 will be a starting face for the merge operator because it is labelled *closed*. Also, a face node set will be initialized to include f_1 only. During the search, faces f_2 , f_3 , f_4 and f_5 will be visited and added to the face set. Again, as the definition for type 1 depressions states, the search will only be continued to a depth of 1, and only external concave edges will be considered. Therefore, the face node set $\{f_1, f_2, f_3, f_4, f_5\}$ is classified as a type 1 depression.

For two additional examples, refer to the part depicted in Figure 5. Faces f_1 , f_2 , f_3 , f_4 , f_5 and the two faces of the stepped feature are labelled *bounded*. When the search for type 2 depressions is performed, any of these faces can be used as a starting face. Without loss of generality, assume that f_3 is used as a starting face. The merge operator will traverse the OFAG in a depth-first manner starting at f_3 . Faces f_1 , f_2 , f_4 and f_5 will be visited and added to the face node set that originally contained only

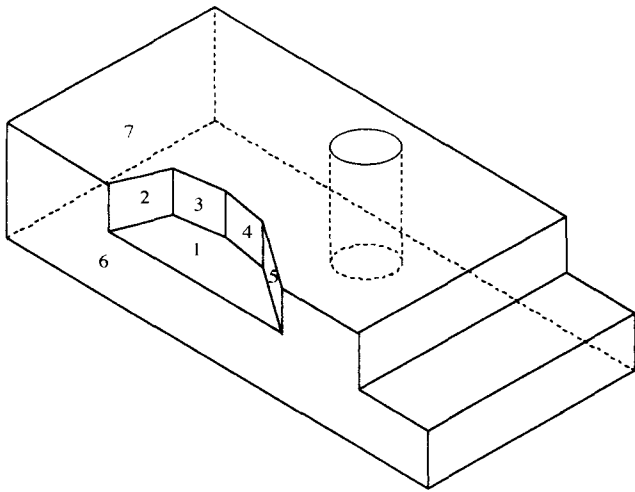


Figure 5 Example solid containing multiple features

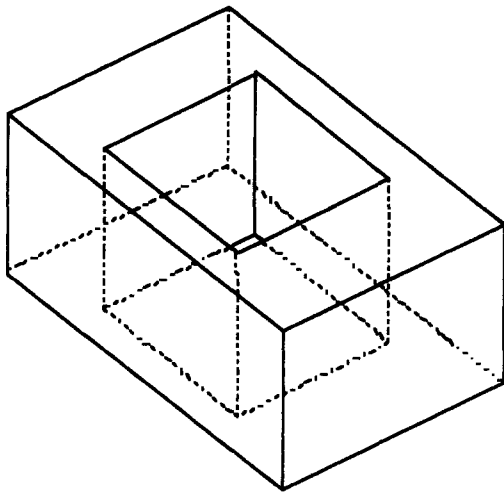


Figure 6 Object with square passage

f_3 . This is true because the definition for type 2 depressions states that face nodes labelled *bounded* and edges labelled *concave* and *exterior* should be considered during the search. Therefore, the resulting face set $\{f_1, f_2, f_3, f_4, f_5\}$ represents a type 2 depression. The two faces of the stepped shape on the end of the part will also be classified as a type 2 depression by this constructive definition. Finally, note that the side faces of the type 1 depression depicted in Figure 3 and described above will not form a type 2 depression. This is true because each of these faces has a concave adjacency to a face node that is not labelled *bounded*.

In many practical situations, it might be most appropriate to group all depression type features into a single class of generalized pockets, sometimes referred to as virtual pockets³². The reader is referred to the sixth section for a discussion of this and some examples.

Another prominent machining domain feature type is the hole. In this work, holes are referred to as passages. A passage is a convex void through an object that usually, but not necessarily, increases the genus of the associated boundary graph. For example, the cylindrical through hole depicted in Figure 5 and the square through hole depicted in Figure 6 will be recognized as passages. Most

often, the sides of the passage will form a cylinder. The definition follows.

Definition 9 (passage): A passage is a subset of the faces of the oriented face adjacency path that is formed by applying the merge operator starting at a through face, specifying an arbitrary recursive depth, and considering only through faces and concave external edges. Also, it is required that there exist no concave edges incident from a face in the merged face set to a face not in the merged face set.

Refer to Figure 6 for an example. The four side faces of the square hole will be labelled *through*. When the search for passages is performed, any of these faces can be used as a starting face. The merge operator, traversing only concave external edges, and recursing from through faces only, merges all these side faces into one set. This set represents the passage feature.

Protrusions are prominent machining domain feature types. Intuitively, a protrusion is defined to be a volume projecting from within a face of the solid. An example might be a cylindrical boss. The definition follows.

Definition 10 (protrusion): A protrusion is a subset of the faces of the oriented face adjacency graph formed by applying the merge operator starting at a bounded face, specifying an unlimited recursive depth, and considering only bounded faces and convex or concave external edges. Additionally, it is required that all the bounded face nodes in the resulting face node set have a concave and internal adjacency with one common face. That is, a protrusion feature must be located within a face of the part. This common face is also defined to be part of the resulting face node set.

For example, refer to Figure 7. The sides of the protruding volume will all be labelled *bounded*. During the search for protrusions, any of these faces can be used as a starting face. The merge operator, considering only bounded faces and external edges, will place all these bounded faces into a face node set. Since all these faces are concave and internally adjacent to one common face, these side faces and the common internal face are classified as a protrusion feature.

These feature classifications are summarized in Table 1. Note that it is possible to introduce other face and edge labelling schemes and in turn to define additional

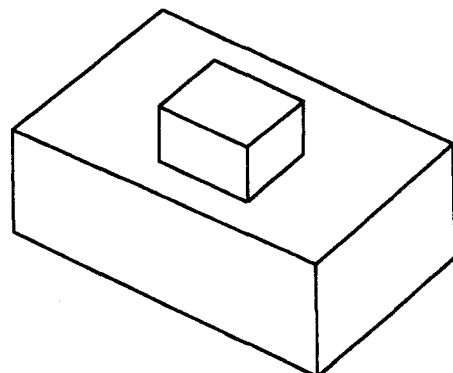


Figure 7 Part with simple protrusion

Table 1 Summary of feature classifications

Feature name	Start face type	Secondary face type	Edge type	Recursive search depth	Special rules
Type 1 depression	Closed	Bounded	Concave, external	One	Yes
Type 2 depression	Bounded	Bounded	Concave, external	Unlimited	Yes
Type 3 depression	Through or bounded	Bounded	Concave, external	One	Yes
Passage	Through	Through	Concave, external	Unlimited	Yes
Protrusion	Bounded	Bounded	Concave or convex, external	Unlimited	Yes

feature types using these labelling schemes and the search operator. In this way, it might be possible to extend these methods to handle additional geometry and shape classification.

RECOGNITION ALGORITHMS

In this section, algorithms are presented to recognize the features described by the feature definitions of the previous section. Formal asymptotic complexity results are presented using concepts and notation that is common to theoretical computational research. The reader who is not familiar with these concepts or notation is referred to Reference 33. The reader is also referred to Reference 34 for an introduction to depth-first searching and other linear graph algorithms. The main goal is to present the algorithms completely and to show the overall efficiency of the feature search method. To this end, experimental timing results based on the computer system hardware clock are presented in the next section.

It is assumed that the convexity or concavity of the intersection of two faces can be computed in constant time. If the faces are planar, cylindrical, or spherical, then this is a simple task. However, if one or both of the faces is of arbitrary curvature then this task becomes more complicated; recall that the intersection must be concave or convex uniquely along the entire intersection.

In the rest of this section, let n designate the total number of edges in the oriented face adjacency graph, and let m designate the number of faces. Note that there are two edges in the OFAG for each edge in the original B-rep. Also note that, since only manifold solids are considered, the number of edges in the OFAG bounds the number of faces. Therefore, most of the overall complexity arguments in this section will be stated in terms of the number of edges only.

Finally, before beginning the discussion of the feature recognition search algorithms, it should be noted that the search for each feature type is performed independently and without regard to the other feature types being considered. Because of this, each face is considered separately during each of the five search algorithms. That is, each face is treated equally during each feature search. Also, because each algorithm is independent of the other search algorithms, the complexity of each algorithm is analysed separately.

To begin with, the search for type 3 depressions is considered. This search consists of two phases. Phase I searches the oriented face adjacency graph for candidate starting faces. When a starting face is found, phase II is invoked. This consists of the merge operation. The complete algorithm is given below.

Algorithm Type 3_Depressions

```

type 3_phase_I
    set all faces to unmarked
    for all faces  $f$  do
        if  $f$  is unmarked and is labelled
            through or unbounded
        then
            execute phase_II( $f$ )
        endif
    endfor
end phase_I

type 3_phase_II( $f$ )
    merge_count = 0
    initialize the feature face set to
        include  $f$  only
    mark  $f$ 
    for all unmarked faces  $f'$  externally
        adjacent and concave to  $f$  do
            if  $f'$  is labelled bounded
            then
                add  $f'$  to the feature
                    face set
                mark  $f'$ 
                merge_count =
                    merge_count + 1
            endif
        endfor
    if merge_count > 0
    then
        output face set and the
            associated edges as a feature
    endif
end phase_II
end Algorithm Type 3_Depressions

```

The time spent in phase I sums to only $O(m)$ time overall because there are only m faces. Now note that the search performed in phase II will never begin from a face that has already been marked, and will never continue from a face that has already been marked. Also note that an edge incident from a face f can only be traversed once before f is marked. Therefore, over all the invocations of phase II, no edge is traversed more than once. Phase II will in the worst case only be invoked m times. Thus the sum of the number of steps required by all the invocations of phase II is bounded by a constant multiple of $n + m$. Therefore, because the number of edges always bounds the number of faces, the overall complexity of the algorithm for computing type 3 depressions is $O(n)$. Type 1 depressions can be computed in a similar fashion.

Searching for type 2 depressions is more complex. The search consists of three phases. Phase I begins by

searching for a starting face labelled *bounded*. When a starting face is found, phase II begins. This performs the merge operation considering only bounded faces and external concave edges. Phase II will form a connected component of the oriented face adjacency graph with respect to these edges and faces. When the search is completed, phase III is employed.

Phase III checks whether there is a bounded face in the merged face node set that is adjacent to all other bounded faces, and it checks that this face is not adjacent and concave to a face having a label other than *bounded*. This check can be performed by sorting the faces which are not concave to nonbounded faces, keying on the number of concave adjacencies of the face. If the face with the highest number of concave adjacencies is not adjacent to every other face in the merged face node set, then the face node set made up of the face with the highest number of concave adjacencies and all the bounded faces adjacent to it is output. This 'splitting' of the face node set is performed with respect to the remaining bounded face nodes with the highest number of concave adjacencies until all the faces in the merge face node set are output. It is possible that an overlap or intersection exists in the output face node sets. In general this is true; when feature shapes intersect, some faces must be included in more than one feature face set.

Note that phase III requires the faces of the merged face node set to be sorted, keying on the number of concave adjacencies. Since the number of concave adjacencies cannot exceed the number of edges incident from the faces in the merged face node set, a bucket sort can be performed in linear time. The splitting can be performed by one traversal of the sorted list. Now note that no face will be included in more than one merged face node set during the overall search for type 2 depressions. Therefore, the number of sorting and traversal steps summed over all invocations of phase III is bounded by a constant multiple of n . Thus phase III requires only $O(n)$ time. From the discussion above, the time required for phase I and phase II is bounded by $O(n)$. Therefore, the algorithm requires only $O(n)$ time overall. The complete algorithm is given below.

Algorithm Type 2_Depressions

```

type 2_phase_I
  set all faces to unmarked
  for all faces  $f$  do
    merge_count = 0 /* global */
    if  $f$  is unmarked and labelled
      bounded
    then
      initialize the face set
        FS to NULL
      mark  $f$ 
      execute phase_II( $f$ )
        returning face set FS
      if merge_count > 0
      then
        execute phase_
          III on the face
            set FS
      endif
    endif
  endfor
end phase_I

```

```

type 2_phase_II( $f$ )
  add  $f$  to the feature face set FS
  for all unmarked faces  $f'$  externally
    adjacent and concave to  $f$  do
    if  $f'$  is labelled bounded
    then
      mark  $f'$ 
      merge_count =
        merge_count + 1
      execute phase_II( $f'$ )
    endif
  endfor
end phase_II

type 2_phase_III
  tag all bounded faces in the face set
    that are adjacent and concave to
    a face that is not labelled bounded
  bucket sort the bounded faces by the
    number of concave adjacencies
  while faces remain in the sorted list do
    let  $f'$  be the face in the list with
      the highest count
    if  $f'$  is not tagged
    then
      output  $f'$  and all the
        bounded faces
        concave to  $f'$  as a
        feature
      delete these faces from
        the sorted list
    endif
  endwhile
end phase_III
end Algorithm Type 2_Depressions

```

Searching for protrusions is similar to searching for type 2 depressions. Phase I searches for appropriate starting faces. Phase II performs the merge operation considering only bounded faces and external edges. Phase II checks whether all the bounded faces in the merged face node set are internally adjacent and concave to one common face; this common face is referred to as the bottom face of the protrusion face set. Faces in the merged face set that are not internally adjacent to a common face are not part of the protrusion feature face set. Let k designate the number of edges incident from the faces in the resulting face node set. Phase III can be performed in $O(k)$ time for any one protrusion by visiting, only a constant number of times, every concave edge of every bounded face in the merged face node set. Since no face is included in more than one protrusion face node set, and thus no edge will be visited more than once, the overall processing requirements for all invocations of phase III sum to a constant multiple of n . From the previous discussion, it can be seen that the processing time required for phase I and phase II is bounded by $O(n)$. Therefore, the algorithm requires only $O(n)$ time overall.

Phase I of the search for passages searches for appropriate starting faces. Phase II performs the merge operation considering only through faces and concave external edges. In phase III, the concave edges incident

from the faces in the resulting face node set are checked to see if they are incident to a face that is not in the merged face node set. If a nonset face exists that is concave to a face node in the set, then the face node set is discarded. Since a face can be included in only one passage feature, the overall processing requirements for all invocations of phase III sum to a constant multiple of n . Therefore, because phase I and phase II require only $O(n)$ time, the recognition of passages requires only $O(n)$ time overall.

Therefore, because the search for each feature type requires only $O(n)$ time, and because there is only a constant number of feature types, the recognition algorithms require only $O(n)$ time overall.

EXPERIMENTAL RESULTS

We have developed FX, a prototype feature extraction implementation based on the concepts and algorithms presented. The experiments presented in this section were performed on a Silicon Graphics workstation. The data is derived from the system clock, which has an accuracy of 10 ms. For each example, three timing values are

supplied: (a) the processing time required to read the boundary data from the file system and construct the oriented face adjacency graph, (b) the processing time required to label the adjacencies and build the face adjacency matrix, and (c) the actual processing time required by the feature recognition algorithms.

The part depicted in *Figure 8* is referred to as the Gehaeuse¹. The Gehaeuse was first used at the CAM-I Geometric Modeling Seminar in Bournemouth, UK, to provide a benchmark for solid modelling systems.

FX recognized 13 features on the surface of this part. The bottom faces of three different depressions that are recognized by FX are highlighted in *Figure 8*. The highlighted face labelled as face 1 is the bottom face of a type 2 depression with five side faces. Note that this depression feature contains a hole within its bottom face, and that it includes faces from two of the rectangular projections. The face labelled face 2 is the bottom face of a type 3 depression. Note that the sides of this depression include some of the faces of the oblong cylindrical projection as well as some of the faces from all four of the rectangular projections. Two of these faces are shared by the depression characterized by face 1 as the bottom. In practical terms, it might be more appropriate to refine these features or faces on the basis of additional specific machining heuristics. Finally, face 3 is the bottom of a slot feature, that is, a type 3 depression with two sides.

As depicted in *Figure 8b*, five passages are recognized by FX. Passages 4, 5 and 6 are cylindrical. Both of the oblong passages, labelled 7 and 8, have two cylindrical faces and two planar faces. Note that these features, as is the case in general, are recognized even if only a polygonal approximation to the part is available.

Figure 9 depicts a bottom view of the Gehaeuse. Two steps, with candidate bottom faces labelled 9 and 10, are recognized adjacent to the oblong cylindrical boss. Note that these faces, as well as other faces of the oblong cylindrical boss, are used as sides in the depression feature with a bottom face labelled 2 (see *Figure 8*). This demonstrates a case in which one design feature, namely the oblong cylindrical boss, will be recognized as multiple machining oriented features. Also, in general, it is possible for a single face to be used by multiple features. When

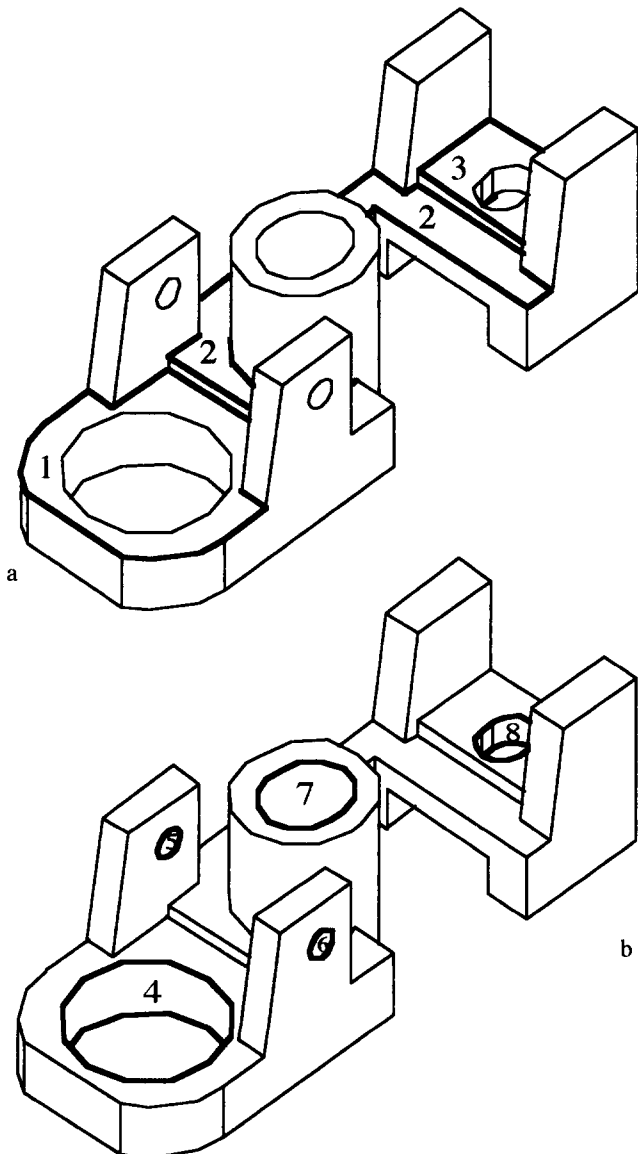


Figure 8 Gehaeuse

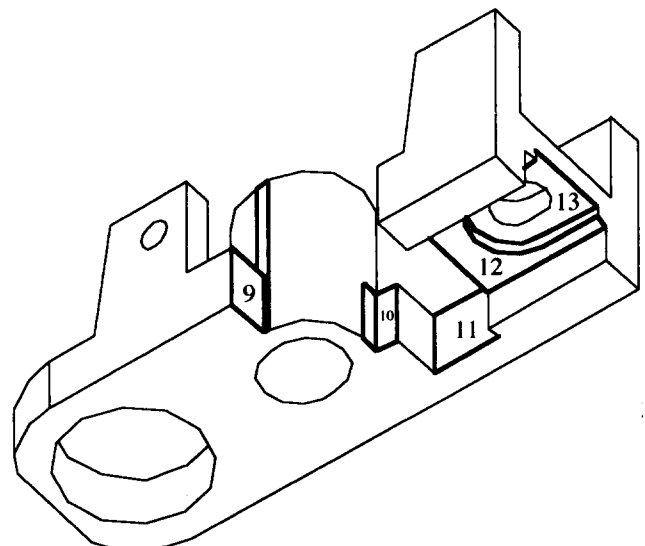


Figure 9 Bottom view of Gehaeuse

this happens, surface blending is required, which is significant when considering the process planning of machining operations. Additionally, two slots, whose bottom faces are labelled 11 and 12, are also recognized. Finally, a type 2 depression is recognized whose bottom face is labelled 13.

The Gehause, as modelled here, has 128 faces and 378 edges. Reading the boundary description from the file system took 200 ms. The data structures took 40 ms to label. This included classification of the convexity or the concavity of the edges of the oriented face adjacency graph and the computation of the adjacency matrix. Note that all required floating point computations are performed in the labelling phase. FX recognized all the features as described above in 40 ms. Note that more computation time is required to compute and label the oriented face adjacency graph than is required for feature recognition. The total time taken to input, label and compute the features of the Gehause was 280 ms.

Finally, a part that was constructed from a large number of faceted passages, depressions and protrusions was examined. This part had 4159 faces and 11 712 edges. Approximately 8.830 s were required to input the boundary data structure, and 4.120 s were required to label the data structures. The 325 features, which FX recognized, required 2.340 s to compute.

In conclusion, we have tried to demonstrate the robustness, generality and speed of these methods and algorithms. The part depicted in *Figures 8 and 9* demonstrates that the methods are sufficiently general to perform well for complex prismatic parts. The overall processing times for this part, at less than 0.5 s, demonstrate that, when these methods are used, general feature recognition can be carried out at interactive speeds.

CONCLUSIONS AND FUTURE WORK

There is a spectrum of feature recognition methods in the literature. These all have the following form: (a) a classification of feature shapes is defined or a method is provided to allow features to be enumerated, (b) the recognition algorithms to search for and recognize these features are defined, (c) it is argued explicitly or implicitly that these shapes correspond to the generally accepted notion of features with respect some application domain. All the previous methods have had difficulties with either coverage, efficiency, or analysability.

The methodology presented in this paper attempts to overcome some of these weaknesses. Search algorithms for protrusion, depression and passage features are presented that not only recognize the standard machining oriented features, such as pockets, slots, steps and holes, but also recognize many of the arbitrarily shaped features that result when standard features intersect. Research is continuing so that the coverage of the recognition algorithms can be expanded to include other features in manufacturing and other domains.

The efficiency of the search algorithms is also important. The analysis of the computational complexity of the algorithms, which resulted in a linear time bound, is supported by empirical data. Recall that less than 0.5 s of processing time is required to recognize all the features of a complicated part such as the Gehause.

The feature descriptions and algorithms presented are

not without limitation. Machining-oriented features that commonly occur in prismatic machined parts are described and are quickly recognized by these methods. No attempt has been made to describe or recognize features that are not machinable by the standard methods. However, taking a practical machining-oriented point of view, and considering the speed and generality of the resulting methodology, we feel that the advantages of the methodology far outweigh the limitations.

REFERENCES

- 1 Veenman, P 'Romulus — the design of a geometric modeler' *Pres. CAM-I Geometric Modelling Sem.* (27 Nov 1979)
- 2 *Webster's Ninth New Collegiate Dictionary* Merriam-Webster, USA (1983)
- 3 Dixon, J and Cunningham, J 'Designing with features: the origin of features' *Proc. ASME Computers in Engineering Conf.* San Francisco, CA, USA (1988)
- 4 Garey, M and Johnson, D *Computers and Intractability: A Guide to the Theory of NP-Completeness* W H Freeman (1979)
- 5 Sugihara, K 'An $n \log n$ algorithm for determining the congruity of polyhedra' *J. Comput. & Syst. Sci.* Vol 29 No 1 (1984)
- 6 Hopcroft, J and Tarjan, R 'A $V \log V$ algorithm for isomorphism of triconnected planar graphs' *J. Comput. & Syst. Sci.* Vol 7 (1973) pp 323–333
- 7 Joshi, S and Chang, T 'Graph-based heuristics for recognition of machined features from a 3D solid model' *Comput.-Aided Des.* Vol 20 No 2 (1988) pp 58–66
- 8 Henderson, M R and Chuang, S H 'Three-dimensional shape pattern recognition using vertex classification and vertex-edge graphs' *Comput.-Aided Des.* Vol 22 No 6 (1990) pp 377–386
- 9 Gossard, D and Sakurai, H 'Recognizing shape features in solid models' *IEEE Comput. Graph. & Applic.* Vol 10 No 5 (1990) pp 22–32
- 10 Henderson, M 'Extraction of feature information from three dimensional CAD data' *PhD Thesis* School of Mechanical Engineering, Purdue University, USA (May 1984)
- 11 Kyprianou, L K 'Shape classification in computer aided design' *PhD Thesis* Christ's College, University of Cambridge, UK (1980)
- 12 Prinz, F, Pinilla, J and Finger, S 'Shape feature description and recognition using augmented topology graph grammar' *Proc. NSF Engineering Design Research Conf.* (1989) pp 285–300
- 13 De Florian, L 'Representation and extraction of shape features in a solid model' in *NATO ASI of Theoretical Foundations of Computer Graphics and CAD* (1987)
- 14 Gavankar, P and Henderson, M R 'Graph-based extraction of protrusions and depressions from boundary representations' *Comput.-Aided Des.* Vol 22 No 7 (1990) pp 442–450
- 15 Corney, J and Clark, D E R 'Efficient face-based feature recognition' *Proc. Solid Modeling '93 Conf.* Montreal, Canada (May 1993)
- 16 Corney, J and Clark, D E R 'Face-based feature recognition: generalizing special cases' *Int. J. Comput. Integr. Manuf.* Vol 6 No 1/2 (1993) pp 39–50
- 17 Corney, J, Clark, D E R, Murray, J L and Yue, Y 'Automatic classification of 2 1/2 D components' *Proc. Concurrent Engineering 1992 ASME, USA* (1992) (PED Vol 50)
- 18 Corney, J and Clark, D E R 'Method for finding holes and pockets that connect multiple faces in 2 1/2D objects' *Comput.-Aided Des.* Vol 23 No 10 (1991) pp 658–668
- 19 Henderson, M R, Anderson, D C and Staley, S M 'Using syntactic pattern recognition to extract information from a solid geometric data base' *Comput. Mech. Eng.* Vol 2 No 2 (1983) pp 61–68
- 20 Falcidieno, B and Giannini, F 'Extraction and organization of form features into a structured boundary model' *Proc. Eurographics Amsterdam, Netherlands* (1987)
- 21 De Florian, L 'Extraction of topological features from a surface based model of a solid object' Cappellina, V and Marconi, R (Eds.) *Advances in Image Processing and Pattern Recognition* North-Holland (1985)
- 22 De Florian, L and Falcidieno, B 'A hierarchical boundary model for solid object representation' *Technical Report 209 IMA*, Genova, Italy (1986)
- 23 Jared, G E 'Shape Features in Geometric Modeling' in *Solid Modeling by Computers: From Theory to Applications* Plenum Press, USA (1984)

- 24 Falcidieno, B, Ansaldi, S and De Floriani, L 'Geometric modeling of solid objects by using a face adjacency graph representation' *Comput. Graph.* Vol 19 No 3 (1985) pp 131–139
- 25 Vandenbrande, J 'Automatic recognition of machinable features in solid models' *PhD Thesis* Department of Computer Science, University of Southern California, USA (1990)
- 26 Marefat, M and Kashyap, R L 'Geometric reasoning for recognition of three-dimensional object features' *IEEE Trans. Pattern Anal. & Mach. Intell.* Vol 12 No 10 (1990) pp 949–965
- 27 Wang, M T 'A geometric reasoning methodology for manufacturing feature extraction for a 3-D CAD model' *PhD Dissertation* Department of Industrial Engineering, Purdue University, USA (1990)
- 28 Woo, T C *Computer Aided Recognition of Volumetric Designs* North-Holland, Netherlands (1977)
- 29 Kim, Y S 'Recognition of form features using convex decomposition' *Comput.-Aided Des.* Vol 24 No 9 (1992)
- 30 Waco, D L and Kim, Y S 'Geometric reasoning for machining features using convex decomposition' *ACM Symp. Solid Modeling & Applications* Montreal, Canada (1993)
- 31 Ferreira, J C E and Hinduja, S 'Convex-hull-based feature-recognition method for 2.5D components' *Comput.-Aided Des.* Vol 22 No 1 (1990) pp 41–49
- 32 Chang, T *Expert Process Planning for Manufacturing* Addison-Wesley (1990)
- 33 Aho, A V, Hopcroft, J E and Ullman, J D *The Design and Analysis of Computer Algorithms* Addison-Wesley (1974)
- 34 Tarjan, R 'Depth-first search and linear graph algorithms' *SIAM J. Comput.* Vol 1 No 2 (1972) pp 146–160

BIBLIOGRAPHY

Regli, W C and Nau, D S 'Building a general approach to feature recognition of material removal shape element volumes (MRSEVs)' *Proc. ACM Solid Modeling '93 Conf.* Montreal, Canada (May 1993)

Lu, K S, Srinivasan, R and Liu, C R 'Extraction of manufacturing details from geometric models' *Comput. & Indust. Eng.* Vol 9 No 2 (1985) pp 125–133

Woodward, J 'Some speculations on feature recognition' *Comput.-Aided Des.* Vol 20 No 4 (1988) pp 189–196



Malcolm C Fields is a senior engineer in the Advanced Manufacturing Department at United Technologies Carrier in Syracuse, NY, USA. He is involved in the Knowledge-based Engineering Initiative, and is responsible for the research and development of a knowledge-based mechanical design system. His research has focused on feature recognition from solid geometric models and computational geometry. He recently received a PhD in computer sciences

from Purdue University, USA.



David C Anderson is a professor of mechanical engineering and computer sciences at Purdue University, USA. He conducts research and teaches in the areas of computer-aided design, computer graphics, and mechanical engineering design. His research has focused on problems of computer-aided engineering design environments, integrated CAD/CAM systems, high-level shape representation (geometric modelling) and communications (geometric reasoning) issues.

metric reasoning) issues.